Final Project Guidelines

1. What we are looking for

> **Your final project should be an investigation, not a demonstration of a technique.**

A strong final project begins with an interesting problem, uncertainty, or unanswered question. Machine learning is the tool you use to investigate it. We expect you to go beyond reproducing a class technique on a dataset and reporting the result.

- Ask a meaningful question whose answer is not obvious before you run the experiments.
- Use existing techniques when appropriate, but push them further through new questions, ablations, extensions, robustness tests, improvements, or deeper analysis.
- Aim for a project that could teach another ML practitioner something useful about the problem, method, data, or failure modes.
- Be ambitious in the question but keep execution staged and disciplined.

2. What is not sufficient

The following can be components of a project, but they are not sufficient on their own:

- “Apply method X to dataset Y” and report accuracy / F1 / RMSE.
- Train several standard models and declare the one with the best metric the winner.
- Reproduce a tutorial, Kaggle solution, or paper implementation without asking a new question.
- Tune hyperparameters until performance improves, without analyzing why.

WEAK PROJECT FRAMING	STRONGER INVESTIGATION
Compare random forest, SVM, and XGBoost on dataset X.	When and why do the models fail on X? Are the failures driven by class imbalance, distribution shift, missingness, or feature interactions? Which interventions actually help?
Use transfer learning for image classification.	How does transfer learning behave as labeled data becomes scarce or the target domain shifts? Which layers or representations transfer, and where does it break?
Reproduce paper Y.	Reproduce the core result, then test an assumption, ablate a component, examine a failure mode, or evaluate a setting the paper did not study.

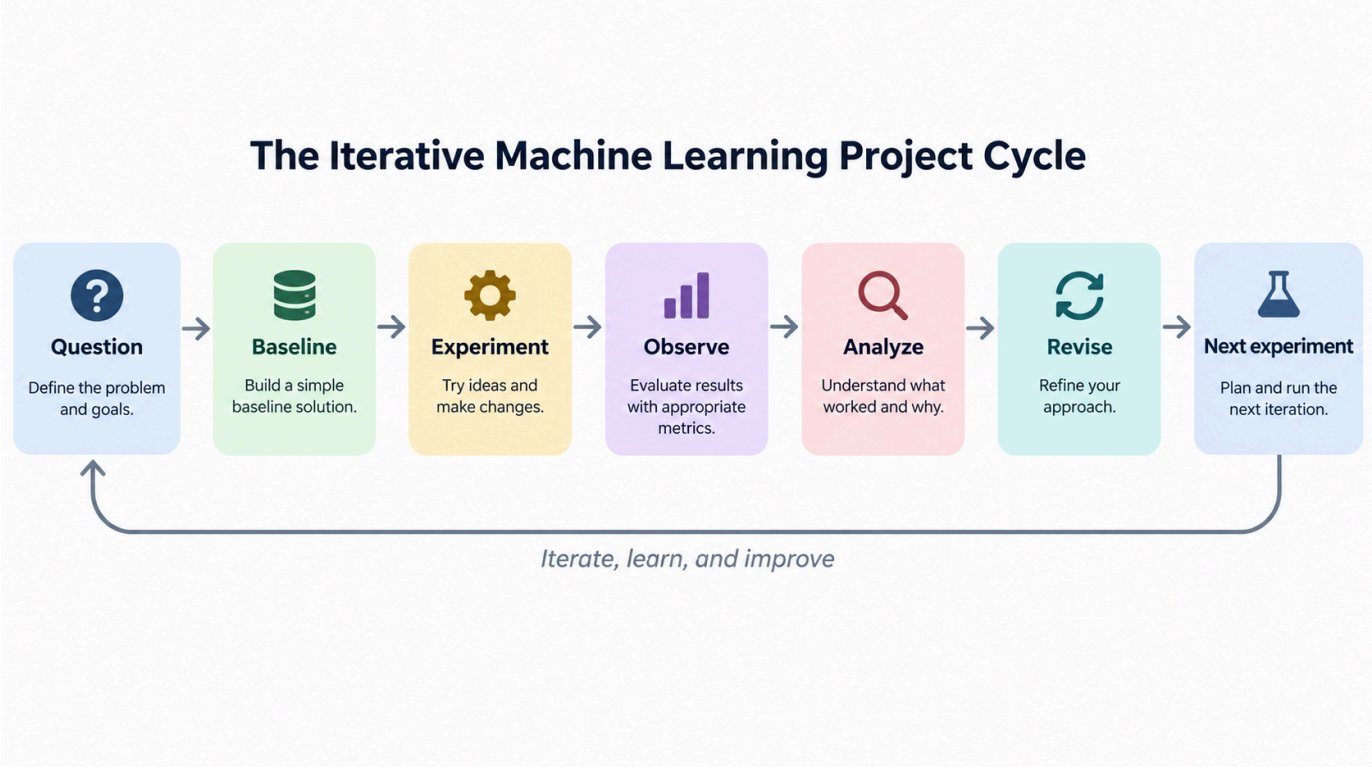
3. Project Guidelines

The final project you deliver should contain the following:

1. **Problem Statement & Motivation**
   Clearly define the problem you are trying to solve or understand. Why is it important, interesting, or technically challenging? What is the guiding question, hypothesis, or objective?
2. **Related Work**
   Summarize the most relevant existing work. What has already been tried? What limitations, gaps, or unanswered questions motivate your project?
3. **System Design & Scope**
   Describe the end-to-end system or approach you plan to build and investigate.
   Define:
   - **Core Goal / MVP:** What is the minimum meaningful version of the project that your team commits to completing?
   - **Stretch Goal(s):** What more ambitious capabilities, extensions, or research directions will you pursue if the core system is successful?
   - **System Design:** What are the major components, and how do they interact? Show the architecture/data flow where appropriate.
   - **Proposed Techniques:** What ML techniques, models, algorithms, or approaches do you expect to try within the system? Why are they reasonable starting points?
4. Your system and scope may evolve as you learn from experiments. The goal at this stage is to establish a thoughtful starting design and a realistic but ambitious scope.
5. **Evaluation Pipeline**
   Define how you will evaluate both the overall system and individual design choices. Include:
   - datasets and evaluation sets;
   - baseline(s);
   - quantitative and/or qualitative metrics;
   - error and failure analysis;
   - ablation studies;
   - robustness, edge-case, or subgroup analysis where relevant.
6. **Experimental Plan**
   Describe the major experiments you plan to conduct. For each experiment, explain **what question it is intended to answer**, rather than simply listing models or techniques you will try.
7. **Investigation & Iteration Plan**
   Explain how you will use experimental results to refine the project. What assumptions are you testing? How will you investigate unexpected results or failures? What might cause you to change your approach?
8. **Challenges & Mitigation**
   Identify the main technical, data, modeling, computational, or evaluation challenges you anticipate. Explain how you plan to address them and what alternatives you will consider if the initial approach does not work.
9. **Expected Learning**
   What do you expect to learn or understand through the project? What insights could make the project valuable even if the proposed approach fails or does not outperform the baseline?
10. **Future Directions**
    Based on what you expect to learn, what would be the most meaningful next steps beyond the scope of this project? Identify promising extensions, unanswered questions, or improvements that could be pursued with more time, data, compute, or engineering effort.

4. Execute in stages and learn from results

Do not spend the semester building one large system and evaluate it only at the end. Work iteratively, build small, test and evaluate, improve based on the results you observe. That's how you will learn.



Here are milestones that can help you execute iteratively:

Milestone	Due date	What teams submit	Main purpose
1. Topic & Problem Framing	Sept 23	Problem statement, motivation, guiding question/hypothesis, initial related work, possible data/resources	Make sure the project starts from an interesting problem, not from a technique to apply. This will not be graded. We will provide feedback to teams.
2. Project Proposal	Oct 14	Problem Statement & Motivation, Related Work, System Design & Scope, Evaluation Pipeline, Experimental Plan, Investigation & Iteration Plan, Challenges & Mitigation, Expected Learning, Future Directions	The Teaching team (Instructor and TAs) will validate that the project is ambitious, well-scoped, evaluable, and has a meaningful investigation plan. This will not be graded. We will provide feedback to teams.

3. MVP + evaluation pipeline Checkpoint	Oct 28	Working core/MVP, data and evaluation pipeline, baseline(s), first experiment(s), preliminary results	Confirm that the team has an end-to-end system working and has started generating evidence rather than spending the whole project on setup. This will not be graded. We will provide feedback to teams.
4. Investigation & Iteration Checkpoint	Nov 11	Experiments completed so far, ablations, failure/error analysis, unexpected findings, challenges encountered, changes to original design, next experiments	We will check whether the team is learning from results and iterating, rather than simply trying more models. This will not be graded.
5. Final Project	Dec 1	Final pdf report, recorded demo, codebase	Evaluate the quality of the investigation, technical execution, analysis, and learning We will not simply focus on the final metric.

5. Failure is acceptable. Shallow analysis is not.

> **Grades are not based on absolute model performance.**
>
> An ambitious idea that fails can be an excellent project if the team conducts a rigorous investigation, identifies what failed and why, and extracts useful learning.

You are expected to show your attempts, not hide the unsuccessful ones. We celebrate failures. In the final report, discuss:

- What did you try that did not work?
- What unexpected results (good or bad) occurred?
- Which assumptions turned out to be wrong or incomplete?
- What did error analysis reveal?
- How did these findings change your next experiment?
- What would you try next with more time or resources?

6. How the project will be evaluated

Evaluation will prioritize the quality of the investigation over the final score of the model. Strong projects demonstrate:

- a meaningful and appropriately ambitious problem;
- clear hypotheses or experimental questions;
- sound baselines and technically rigorous experimentation;

- depth of analysis and interpretation;
- iteration based on evidence;
- honest treatment of failures, limitations, and uncertainty; and
- clear communication of what the team learned.

A better metric does not mean a better project.

A strong project asks a good question, designs credible experiments, learns from what happens, and communicates the insight clearly.